

INDENG 231 Course Project 2

LLM-Powered Personal Knowledge Base on
AI Social Platforms for Shared Human Experiences

1. What Was Built

This project implements a Karpathy-style personal knowledge base using the Anthropic Claude API. The system has three layers: (1) a corpus of raw source documents on the project topic, (2) an LLM-powered compiler that synthesizes the raw sources into structured, interlinked wiki articles, and (3) a retrieval-augmented Q&A; interface that answers questions by grounding responses in the compiled wiki.

1.1 Topic

The knowledge base covers the design of an AI-powered social platform for shared human experiences. The platform connects users who have navigated similar life challenges (illness, grief, career disruption, immigration) on a mentor-mentee model, mediated by an AI conversational interface. The knowledge base synthesizes five research areas: shared experience platform design, mentor-mentee matching algorithms, affective computing, voice dialogue systems (STT/TTS), and empathetic AI design.

1.2 System Architecture

The pipeline follows a raw → wiki → Q&A; flow:

Layer	Files	Description
Raw sources	raw/ (10 .md files)	Source documents covering all topic areas; 47,463 characters total
Compiler	compile_wiki.py	Claude Opus 4.7 + adaptive thinking; synthesizes wiki articles from raw sources; adds [[WikiLink]]
Wiki	wiki/ (10 .md files)	Compiled, interlinked articles; one per major concept; pre-generated and version-controlled
Index	wiki/_index.json	Link graph: outgoing/incoming link counts, word counts per article
Q&A interface	qa.py	Keyword retrieval of top-4 relevant articles; Claude Haiku 4.5 generates cited answers

1.3 Raw Sources (10 Documents)

- 01: Shared experience platforms — design principles, privacy, matching quality metrics
- 02: Mentor-mentee matching — embedding-based retrieval, two-stage ranking, cold-start
- 03: Affective computing — emotion recognition, trajectory modeling, distress detection
- 04: Speech-to-text pipelines — Whisper, streaming STT, disfluency handling, privacy
- 05: Text-to-speech synthesis — neural TTS, SSML prosody control, expressive synthesis
- 06: Dialogue systems — end-to-end neural dialogue, state tracking, conversation policy
- 07: Empathetic AI design — validation-before-advice, failure modes, empathy measurement
- 08: Experience storage and retrieval — pgvector, embedding schema, privacy-preserving retrieval
- 09: Platform integration architecture — microservices, latency budget, feedback loop
- 10: LLM knowledge base design — this system's own architecture and design rationale

1.4 Compiled Wiki (10 Articles)

The compiler generates one wiki article per major concept. Each article includes: a Key Concepts section with 5-bullet definitions, a Design Implications section with actionable takeaways, [[WikiLink]] references to related articles (creating a navigable graph), and a See Also section. The 10 compiled articles are: ExperienceMatching, AffectiveComputing, VoiceInterface, DialogueSystem, EmpathyDesign, MentorMenteeSystem, ExperienceStorageRetrieval, PlatformArchitecture, SafetyAndEscalation, and KnowledgeBaseDesign.

Article	Words	Out Links	In Links
AffectiveComputing	462	10	7
DialogueSystem	596	8	7
EmpathyDesign	681	8	7
ExperienceMatching	435	6	6
ExperienceStorageRetrieval	614	9	5
KnowledgeBaseDesign	809	9	2
MentorMenteeSystem	659	12	4
PlatformArchitecture	687	7	6
SafetyAndEscalation	762	6	7
VoiceInterface	581	7	5

High incoming link counts for AffectiveComputing and SafetyAndEscalation (7 each) reflect their centrality in the knowledge domain — nearly every other article references them.

2. Evaluation: What Worked and What to Improve

2.1 What Worked Well

Synthesis quality. Claude added structure, cross-connections, and design implications absent from the raw sources. The compiled wiki is meaningfully better than the sum of its parts — Key Concepts sections distill 500-word raw documents into 5-bullet definitions; Design Implications sections add practical takeaways not in any single source.

WikiLink coherence. The LLM naturally identified conceptual dependencies and created appropriate inter-article links without explicit instruction. AffectiveComputing and SafetyAndEscalation emerged as the most-linked hub articles, correctly reflecting their centrality in the platform design.

Q&A; accuracy. Keyword retrieval correctly identifies relevant articles for all tested questions. Answers cite specific wiki articles and synthesize across multiple sources when relevant (e.g., 'How does the platform respond to a user in crisis?' correctly draws from AffectiveComputing, SafetyAndEscalation, and DialogueSystem).

Modularity. The raw → wiki → Q&A; pipeline is topic-agnostic. Replacing the raw/ directory with notes on any other domain and rerunning compile_wiki.py produces a working knowledge base on the new topic with no code changes.

Pre-compilation strategy. Committing the pre-generated wiki/ directory to the repository means the Q&A; interface is immediately usable — no compilation cost at demo time, no API key needed just to run --list or explore articles.

2.2 Limitations and What to Improve

Keyword retrieval misses semantic matches. A question like 'How does the system detect sadness?' does not overlap with the article name 'AffectiveComputing' by keyword. Replacing keyword retrieval with embedding-based approximate nearest-neighbor search (using the same pgvector pattern described in ExperienceStorageRetrieval) would improve recall for semantically similar but lexically distant queries.

No live incremental compilation. The wiki is a static snapshot. Adding a new raw source document requires manually rerunning compile_wiki.py. A file-watcher daemon that detects changes to raw/ and re-compiles affected articles would keep the wiki current continuously.

Single-hop retrieval. The Q&A; interface retrieves top-4 articles but does not follow WikiLinks to transitively relevant articles. Questions spanning multiple concepts (e.g., 'How does affective computing improve mentor

matching over time?') would benefit from a graph-walk retrieval that follows links from initially retrieved articles.

No web rendering of WikiLinks. The Q&A; interface renders `[[WikiLink]]` as bold text. A web interface (Flask or FastAPI serving a simple HTML frontend) would make these links clickable, enabling wiki-style navigation between articles — the most natural way to explore a knowledge graph.

Limited evaluation rigor. Compilation quality was assessed qualitatively. A more rigorous evaluation would measure information coverage (what fraction of facts in raw sources appear in the wiki), hallucination rate (facts in the wiki not traceable to raw sources), and Q&A; accuracy on a held-out question set with ground-truth answers derived from the raw sources.